Revisiting Segmentation through transformers

Medical Image Computing Course Project

Dhanush Sanjay, Pavan Sai

May 2026

Architecture Overview: Rethinking U-Net

Model: 2D adaptation of TransUNet (Chen et al.)

The architecture transitions from standard per-pixel classification to a **sequence-to-sequence mask classification** paradigm. It encapsulates Transformer self-attention into two key modules:

1. **Transformer Encoder:** Tokenizes image patches from CNN feature maps to capture global, long-range context.
2. **Transformer Decoder:** Refines candidate regions via cross-attention between learnable organ/vessel queries and U-Net features.

Approach: We utilize the **Encoder+Decoder** configuration to balance global context awareness with fine-grained local detail extraction.

Architecture: Hybrid CNN Encoder

Component: ResNet-50 Backbone (Truncated after Layer 3)

- **Input:** Image patch tensor $x \in \mathbb{R}^{B \times C \times H \times W}$ (where $C = in\_channels$).
- **Initial Convolution (Layer 0):**
  - Custom weights to handle 1-channel (green) inputs.
  - Output: $s_0 \in \mathbb{R}^{B \times 64 \times \frac{H}{2} \times \frac{W}{2}}$.
- **Residual Blocks & Skip Connections:**
  - **Maxpool + Layer 1:** $s_1 \in \mathbb{R}^{B \times 256 \times \frac{H}{4} \times \frac{W}{4}}$.
  - **Layer 2:** $s_2 \in \mathbb{R}^{B \times 512 \times \frac{H}{8} \times \frac{W}{8}}$.
  - *Note:* s_0, s_1, and s_2 are preserved and forwarded as skip connections to the CNN Decoder.
- **Final Feature Map (Layer 3):**
  - Yields the deepest spatial features to be fed into the Vision Transformer.
  - Output: $s_3 \in \mathbb{R}^{B \times 1024 \times \frac{H}{16} \times \frac{W}{16}}$.

Architecture: Vision Transformer Encoder (ViTEncoder)

Component: Transformer Encoder operating on CNN feature maps.

- **Input:** High-level CNN feature map $cnn\_out \in \mathbb{R}^{B \times 1024 \times \frac{H}{16} \times \frac{W}{16}}$.
- **Patch Embedding & Tokenization:**
  - A 1×1 convolution projects the 1024 channels to $embed\_dim$ (e.g., 768).
  - The spatial dimensions are flattened into N tokens, where $N = \left(\frac{H}{16}\right) \times \left(\frac{W}{16}\right)$.
  - Output after projection: $\mathbb{R}^{B \times 768 \times \frac{H}{16} \times \frac{W}{16}}$.
  - Output after flattening: $\mathbb{R}^{B \times N \times 768}$.
- **Positional Embedding:**
  - Learnable position embeddings $pos\_embed \in \mathbb{R}^{1 \times N \times 768}$ are added to retain spatial awareness.
- **Transformer Layers:**
  - The sequence $x \in \mathbb{R}^{B \times N \times 768}$ passes through L Transformer layers (LN $\rightarrow$ MSA $\rightarrow$ Res $\rightarrow$ LN $\rightarrow$ MLP $\rightarrow$ Res).
- **Final Feature Map:**
  - Tokens are reshaped back to spatial dimensions: $\mathbb{R}^{B \times 768 \times \frac{H}{16} \times \frac{W}{16}}$.
  - A 1×1 convolution projects it back: $vit\_out \in \mathbb{R}^{B \times 768 \times \frac{H}{16} \times \frac{W}{16}}$.

Mathematical Formulation Of Encoder

1. Patch Embedding & Positional Encoding

Flattened patches $\mathbf{x}_i^p$ are projected into a latent d_{enc}-dimensional space via a linear projection matrix $\mathbf{E}$. Learnable position embeddings $\mathbf{E}^{pos}$ are added to retain spatial information:

$$\mathbf{z}_0 = [\mathbf{x}_1^p\mathbf{E}; \mathbf{x}_2^p\mathbf{E}; \cdots ; \mathbf{x}_N^p\mathbf{E}] + \mathbf{E}^{pos} \tag{1}$$

Where $\mathbf{E}$ *is the embedding projection and* $\mathbf{E}^{pos} \in \mathbb{R}^{N \times d_{enc}}$ *is the position embedding.*

2. Transformer Layers ($\ell = 1, \ldots, L$)

Each layer consists of Multihead Self-Attention (MSA) and Multi-Layer Perceptron (MLP) blocks, equipped with Layer Normalization (LN) and residual connections:

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1} \tag{2}$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell \tag{3}$$

Where $\mathbf{z}_\ell$ *is the encoded image representation at layer* ℓ.

Architecture: CNN Decoder

Component: Cascaded Upsampler with Skip Connections

- **Input:** ViT features projected to decoder width: $x \in \mathbb{R}^{B \times 256 \times \frac{H}{16} \times \frac{W}{16}}$. (After using a Bridge from 768 Channels of previous outputs to 256 channels)
- **Skip Connections:** $[s_0, s_1, s_2]$ from the CNN Encoder.
- **Upsampling Stages (UpBlocks):**
  - **Up1:** Bilinear upsample to $\frac{H}{8} \times \frac{W}{8}$. Concat with s_2 (512 ch). Output: $f_1 \in \mathbb{R}^{B \times 256 \times \frac{H}{8} \times \frac{W}{8}}$.
  - **Up2:** Bilinear upsample to $\frac{H}{4} \times \frac{W}{4}$. Concat with s_1 (256 ch). Output: $f_2 \in \mathbb{R}^{B \times 128 \times \frac{H}{4} \times \frac{W}{4}}$.
  - **Up3:** Bilinear upsample to $\frac{H}{2} \times \frac{W}{2}$. Concat with s_0 (64 ch). Output: $f_3 \in \mathbb{R}^{B \times 64 \times \frac{H}{2} \times \frac{W}{2}}$.
  - **Up4:** Bilinear upsample to $H \times W$ (No skip connection). Output: $f_4 \in \mathbb{R}^{B \times 32 \times H \times W}$.
- **Final Output:** A list of multi-scale spatial features $[f_1, f_2, f_3, f_4]$ passed to the Transformer Decoder.

Architecture: Transformer Decoder

Component: Mask2Former-style Coarse-to-Fine Refinement

- **Input:** Multi-scale features $[f_1, f_2, f_3, f_4]$ from the CNN Decoder.
- **Query Initialization:** $N_q (N_q > K)$ learnable queries $P^0 \in \mathbb{R}^{B \times 2 \times 256}$ (for background and vessel).
- **Initial Coarse Mask:** Dot product of P^0 and projected high-res feature f_4 yields $Z^0 \in \mathbb{R}^{B \times 2 \times H \times W}$.
- **Iterative Refinement (4 Layers):**
  - **Foreground Mask Generation:** Previous mask Z^{t-1} is downsampled to current feature resolution $\frac{H}{2^{4-t}} \times \frac{W}{2^{4-t}}$ and binarized.
  - **Self-Attention:** Queries interact with each other ($P^t \rightarrow P^t$).
  - **Masked Cross-Attention:** Queries attend to feature map f_t, restricted only to regions where the foreground mask is active.
  - **Mask Update:** Updated queries generate the next mask $Z^t \in \mathbb{R}^{B \times 2 \times H \times W}$.
- **Final Output:** Returns final refined masks $Z^4 \in \mathbb{R}^{B \times 2 \times H \times W}$ and auxiliary masks for deep supervision.

Mathematical Formulation Of the Decoder

1. Initial Coarse Mask:

$$\mathbf{Z}^0 = g(\mathbf{P}^0 \times \mathbf{F}^\top)$$

2. Masked Cross-Attention (Eq. 6):

$$\mathbf{P}^{t+1} = \mathbf{P}^t + \text{Softmax}((\mathbf{P}^t\mathbf{w}_q)(\mathcal{F}\mathbf{w}_k)^\top + h(\mathbf{Z}^t)) \times \mathcal{F}\mathbf{w}_v$$

3. Foreground Masking (Eq. 7):

$$h(\mathbf{Z}^t(i,j,s)) = \begin{cases} 0 & \text{if } \mathbf{Z}^t(i,j,s) = 1 \\ -\infty & \text{otherwise} \end{cases}$$

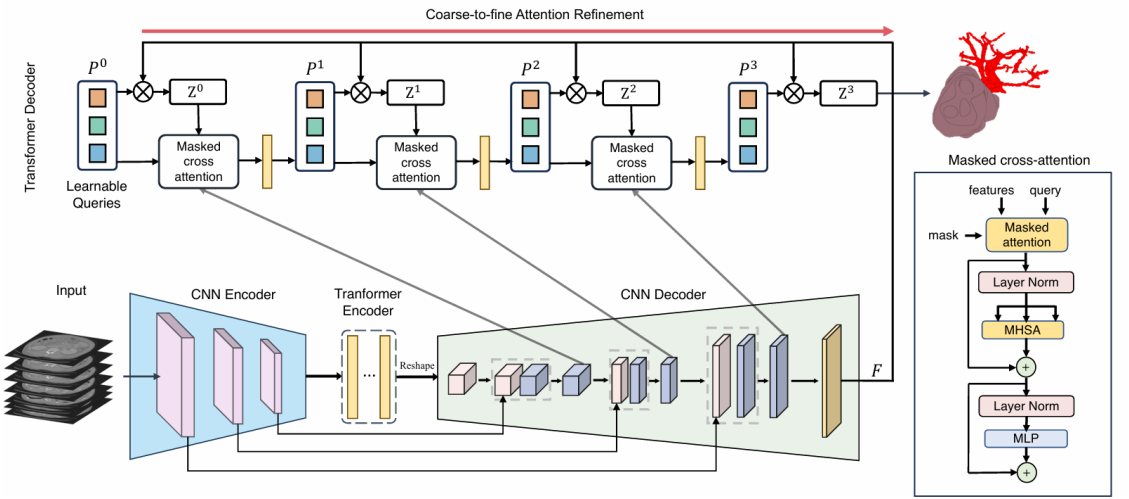
Note: $g(\cdot)$ is sigmoid + hard thresholding. $\mathbf{F}$ *is the highest-res feature,* $\mathcal{F}$ *is the intermediate feature.*

Algorithm 1: Coarse-to-fine Refinement

Input: Queries $\mathbf{P}$, Features $\mathbf{F}, \mathcal{F}$, Iterations T
Output: Fine map $\mathbf{Z}^T$, Labels $\hat{\mathbf{y}}$

$t \leftarrow 0$; $\mathbf{P}^0 \leftarrow \mathbf{P}$
$\mathbf{Z}^0 \leftarrow g(\mathbf{P}^0 \times \mathbf{F}^\top)$
repeat
  Update $\mathbf{P}^{t+1}$ using Eq. 6
  Update $\mathbf{Z}^{t+1} \leftarrow g(\mathbf{P}^{t+1} \times \mathbf{F}^\top)$
  $t \leftarrow t + 1$
until $t = T$
Compute class label $\hat{\mathbf{y}}$
Return: $\mathbf{Z}^T$, $\hat{\mathbf{y}}$

Architecture Visualized



Dataset 1

DRIVE: Digital Retinal Images for Vessel Extraction

Dataset 1: DRIVE Retinal Vessel Segmentation

- **Objective:** Segment retinal blood vessels to aid in diagnosing ocular diseases.
- **Dataset:** DRIVE (Digital Retinal Images for Vessel Extraction).
- **Data Split:** 20 labelled images total $\rightarrow$ 16 for training, 4 for validation.
- **Preprocessing Strategy:**
  - Extracted the **green channel** for optimal vessel contrast.
  - Patch-based training utilizing random crops of size 224×224 mostly within the field of view.
  - Augmentations: Random horizontal/vertical flips and rotations ($\pm 30^{\circ}$).
- **Training Parameters:**
  - **Epochs:** 30
  - **Batch Size:** 8

Architecture: CNN & Transformer Encoders

Hybrid CNN Encoder

- **Backbone:** ResNet-50 (pretrained), truncated.
- **Input:** $x \in \mathbb{R}^{B\times 1\times 224\times 224}$
- **Skip Connections:**
  - s_0: $B \times 64 \times 112 \times 112$
  - s_1: $B \times 256 \times 56 \times 56$
  - s_2: $B \times 512 \times 28 \times 28$
- **Output:** $cnn\_out \in \mathbb{R}^{B\times 1024\times 14\times 14}$

Vision Transformer Encoder

- **Tokenization:** 1×1 conv maps 1024 to 768 dims. Flattened to 196 tokens.
- **Sequence:** $tokens \in \mathbb{R}^{B\times 196\times 768}$ with position embeddings.
- **Layers: 1 Transformer Layer** (optimized for efficiency).
- **Output:** $vit\_out \in \mathbb{R}^{B\times 768\times 14\times 14}$

Architecture: CNN & Transformer Decoders

Cascaded CNN Decoder

- **Bridge:** Maps ViT output to decoder width ($B \times 256 \times 14 \times 14$).
- **Upsampling:** 4 stages merging with skip connections.
- **Multi-Scale Features (f_t):**
  - f_1: $28 \times 28 \times 256$
  - f_2: $56 \times 56 \times 128$
  - f_3: $112 \times 112 \times 64$
  - f_4: $224 \times 224 \times 32$

Transformer Decoder Mechanism

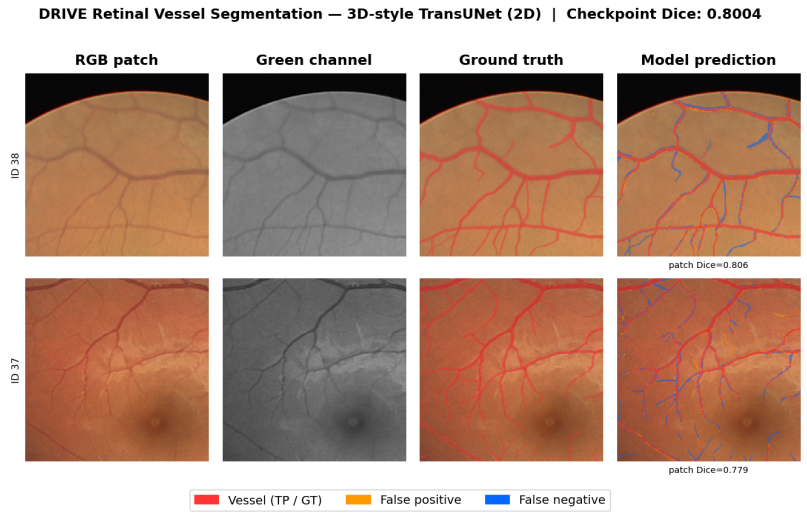
- **Queries:** 2 learnable organ queries initialized to 0.
- **Coarse-to-Fine Refinement:** Masked cross-attention across 4 layers.
- **Mechanism:** Cross-attention is constrained by the foreground of previous coarse predictions.
- **Output:** Final refined logits $(B, 2, 224, 224)$ via dot product.

Training Strategy & Custom Parameters

The model was trained in a Google Colab environment with customized hyperparameters tailored for resource efficiency and optimal convergence:

- **Loss Function:** $\mathcal{L} = 0.5 \cdot \text{BCE} + 0.5 \cdot \text{Dice} + \text{Auxiliary Deep Supervision Loss}$.
- **Optimizer:** AdamW with Cosine Annealing Learning Rate scheduler.
- Mixed Precision (AMP) enabled to fit the batch size of 8 into GPU VRAM.

Results Visualization



Results & Evaluation

Performance on the 4 held-out validation images peaked at **Epoch 9**.

1-Layer Enc + Decoder

Metric / Detail	Value
Dice Coefficient	0.8004
Sensitivity (Recall)	0.8155
Specificity	0.9710
Overall Accuracy	0.9530
Model Parameters	**28.1M**

Other: 12-Layer Enc-Only

Metric / Detail	Value
Dice Coefficient	0.7614
Sensitivity (Recall)	0.7814
Specificity	0.9657
Overall Accuracy	0.9450
Model Parameters	**101.4M**

Conclusion: The Encoder+Decoder TransUNet architecture successfully isolates the vascular structure. The masked cross-attention effectively handles the thin vessel topology, achieving an optimal Dice score of 0.80 while remaining computationally lightweight (**28.1M parameters**) via the 1-layer ViT adaptation, distinctly outperforming the 12-layer Encoder-only baseline.

Dataset 2

BTCV: Beyond the Cranial Vault — 30 abdominal CT volumes

Dataset 2: BTCV Multi-Organ CT Segmentation

- **Dataset:** BTCV (Landman et al., MICCAI 2015) — 30 abdominal CT volumes, 3779 axial slices total.
- **Data Split:** 18 volumes (2212 slices) for training, 12 for validation — following Fu et al. (2020).
- **Preprocessing:**
  - 3D NIfTI volumes ($H \times W \times D$) split into 2D axial slices — slice-by-slice 2D training.
  - CT Windowing: HU clipped to $[-125, 275]$, normalised to $[0, 1]$.
  - Background-only slices skipped during training.
  - Each slice resized to 224×224.
  - Augmentation: random flips and rotations ($\pm 30°$).
- **Classes:** 9 total — background + 8 abdominal organs (aorta, gallbladder, spleen, left/right kidney, liver, stomach, pancreas).
- **Metric:** Average DSC over 8 foreground organs (background excluded).

Architecture: Hybrid Encoder & CNN Decoder

Encoder Adaptations

- **CNN Backbone:** Extracts multi-scale features; skip connections (s_0, s_1, s_2) forwarded to maintain spatial detail.
- **Efficient ViT:** 1-layer Transformer encoder paired with the decoder (Section 4.3). Achieves performance comparable to 12-layer encoders at significantly lower compute.

CNN Decoder

- **Bridge:** Compresses channels ($768 \rightarrow 256$) at 14×14 resolution.
- **Multi-scale Upsampling:** Generates feature maps from 28×28 up to 224×224.
- **Output:** Features are passed to the Transformer Decoder for Coarse-to-Fine (C2F) refinement.

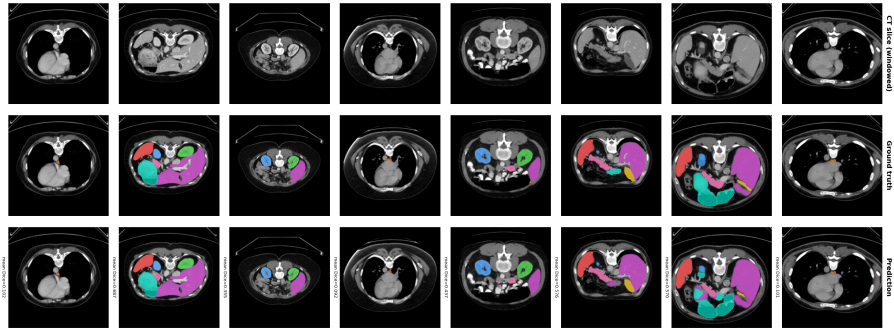
Refinement Mechanism & Training Strategy

- **Transformer Decoder:** Uses $N = 20$ learnable organ queries ($N > K$) to minimize false negatives.
- **Iterative C2F Refinement:** Restricts cross-attention to foreground regions via masking ($h(Z^t)$), refining segmentation over $T = 3$ stages.
- **Loss Function:**

$$\mathcal{L} = \lambda_0(\mathcal{L}_{ce} + \mathcal{L}_{dice}) + \lambda_1\mathcal{L}_{cls}, \quad \lambda_0 = 0.7,\ \lambda_1 = 0.3$$

  Deep supervision is applied at each refinement stage output.
- **Training Config:**
  - **Optimizer:** AdamW with cosine annealing.
  - **Learning Rates:** 3×10^{-5} for backbone; 3×10^{-4} for new layers.
  - **Specs:** Batch size 8, AMP enabled, 100 epochs (best checkpoint at epoch 77).

Results Visualization: BTCV



BTCV Multi-Organ Segmentation — TransUNet (Encoder+Decoder) | Checkpoint mean Dice: 0.8208

Results & Evaluation: BTCV

Per-Organ Dice — Epoch 77

Organ	DSC (%)
Aorta	92.9
Gallbladder	86.1
Spleen	90.7
Left Kidney	79.5
Right Kidney	74.9
Liver	84.4
Stomach	76.0
Pancreas	63.9
Mean	**81.1**

Comparison with paper (Table 2)

Config	Params	Avg. DSC (%)
Enc+Dec (paper, 3D)	41.4M	88.4
Enc+Dec (ours, 2D)	23.8M	**81.1**

Gap attributable to 2D vs 3D training and smaller crop size.

Pancreas hardest (63.9%) due to small size and irregular shape. Aorta easiest (92.9%) due to distinctive tubular structure — consistent with paper Table 2.